

Game Log Pipeline

NestJS + PostgreSQL 기반의 게임 로그 적재 및 조회 프로젝트입니다.

README에는 실행 방법과 검증 흐름 위주로 정리했고, 설계 배경과 구조 판단은 별도 문서에 정리했습니다.

- 설계 문서: [문제 1 설계 문서](#)

1. 실행 전 준비

아래 항목이 먼저 설치되어 있어야 합니다.

- Node.js 20 이상
- npm
- Docker Desktop
- Git

VS Code는 권장 사항이며 필수는 아닙니다.

API 확인과 테스트는 Swagger 문서 기준으로 진행할 수 있어, 별도 VS Code 확장 설치의 필요하지 않습니다.

Windows 기준 예시 명령어는 CMD 기준으로 작성했습니다.

PowerShell에서도 실행은 가능하지만 일부 명령어 표기 방식이 다를 수 있습니다.

2. PostgreSQL 실행

이 프로젝트는 PostgreSQL을 Docker 컨테이너로 실행하는 것을 기준으로 합니다.

로컬에 PostgreSQL을 직접 설치하고 계정/DB를 수동으로 만드는 방식보다, Docker로 실행 환경을 맞추는 편이 검증하기 더 간단하다고 판단했습니다.

현재 서버 코드의 DB 연결 정보는 아래 값을 기준으로 동작합니다.

- host: `localhost`
- port: `5432`
- username: `rush`
- password: `rushpass`
- database: `gamedb`

아래 명령어로 PostgreSQL 컨테이너를 실행합니다.

```
docker run --name gamedb-postgres ^
-e POSTGRES_USER=rush ^
-e POSTGRES_PASSWORD=rushpass ^
-e POSTGRES_DB=gamedb ^
-p 5432:5432 ^
-d postgres:16
```

컨테이너가 정상 실행되면 NestJS 서버는 위 설정으로 DB에 연결합니다.
테이블은 TypeORM의 `synchronize: true` 설정으로 자동 생성됩니다.

실행 중인 컨테이너 확인:

```
docker ps
```

3. 프로젝트 설치 및 실행

3-1. 의존성 설치

깃허브에서 프로젝트를 내려받은 뒤, 루트 디렉터리에서 아래 명령어를 실행합니다.

```
npm install
```

이 과정에서 서버 실행에 필요한 의존성과 Swagger 관련 패키지도 함께 설치됩니다.

3-2. 서버 실행

```
npm run start:dev
```

서버가 정상적으로 실행되면 기본적으로 `http://localhost:3000`에서 확인할 수 있습니다.

3-3. 로그 생성기 실행

서버가 실행된 상태에서 별도 터미널을 열고 아래 명령어를 실행합니다.

```
npm run generator
```

이 명령으로 로그 생성기가 동작하며, 배치 단위로 적재 API를 호출합니다.

4. Swagger / OpenAPI 문서

Swagger 문서는 아래 주소에서 확인할 수 있습니다.

```
http://localhost:3000/api-docs
```

실행 후에는 Swagger에서 등록된 API 목록, 요청 파라미터, 응답 형식을 확인할 수 있습니다.

README에서는 주요 실행 흐름만 정리하고, 상세 API 명세는 Swagger(OpenAPI) 문서를 기준으로 확인하는 것을 전제로 했습니다.

5. 검증 순서

프로젝트 검증은 아래 순서로 진행하면 됩니다.

1. Docker로 PostgreSQL 컨테이너 실행
2. `npm install`
3. `npm run start:dev`로 서버 실행
4. `npm run generator`로 로그 적재 수행
5. Swagger(<http://localhost:3000/api-docs>)에서 적재/조회 API 확인
6. 조회 API를 호출해 집계 결과 확인

6. 적재 → 조회 호출 예시

1) 로그 적재

서버 실행 후 아래 명령어로 로그 생성기를 실행합니다.

```
npm run generator
```

이 명령으로 생성된 로그가 배치 단위로 적재 API에 전송됩니다.

2) 조회 API 확인

로그 적재가 끝나면 Swagger 문서에서 조회 API를 호출해 결과를 확인합니다.

예를 들면 DAU, 매출, 리텐션, 경험치/아이템 관련 통계 API를 순서대로 확인할 수 있습니다.

상세 엔드포인트, 요청 파라미터, 응답 형식은 아래 Swagger 문서를 기준으로 확인하면 됩니다.

```
http://localhost:3000/api-docs
```

7. 설계 대비 변경 사항

전체적인 적재 흐름 자체는 설계 문서와 크게 다르지 않지만, 구현 단계에서 아래와 같이 정리한 부분이 있습니다.

- 설계 문서에서는 수신 시각을 `received_at` 개념으로 설명했지만, 실제 구현에서는 엔티티의 `created_at` 컬럼을 적재 시각으로 사용했습니다.
- 저장 스키마는 설계 단계의 공통 컬럼 구조를 유지하되, 실제 엔티티 기준으로 `character_id`, `session_id`, `channel_id` 컬럼을 포함하도록 구성했습니다.
- 이벤트 상세 데이터는 설계 문서와 동일하게 `payload(JSONB)`에 저장하도록 구현했습니다.
- 실행 및 검증 환경은 클라우드 배포 대신 로컬 재현성을 우선해 PostgreSQL Docker 컨테이너 기반으로 정리했습니다.
- 상세 API 명세는 문서에 모두 풀어 적기보다는, 실제 구현 기준의 Swagger(OpenAPI) 문서에서 확인하는 방식으로 정리했습니다.

8. 가정한 내용

- PostgreSQL은 로컬 Docker 환경에서 실행하는 것을 기준으로 했습니다.
 - 로그 생성기와 서버는 동일한 로컬 환경에서 실행한다고 가정했습니다.
 - 상세 API 스펙은 Swagger 문서를 기준으로 검증한다고 가정했습니다.
-

9. 미구현 / 한계

- AWS 등 클라우드 배포 구성은 포함하지 않았습니다.
현재 프로젝트는 로컬 환경 기준으로만 실행과 검증을 진행했으며, 실제 운영 환경에서의 배포 구성이나 네트워크 설정까지는 다루지 않았습니다.
 - Kafka 같은 메시지 큐는 도입하지 않았습니다.
현재 구조는 HTTP 요청을 받아 바로 DB에 저장하는 방식이므로, 트래픽이 급격히 증가하는 상황에서 요청을 버퍼링하거나 비동기로 흡수하는 데에는 한계가 있습니다.
 - 대규모 운영 환경 수준의 부하 테스트는 별도로 진행하지 않았습니다.
기능 동작과 적재/조회 흐름은 확인했지만, 동시 요청이 크게 늘어나는 상황에서의 처리량, 응답 시간, 안정성은 정량적으로 검증하지 못했습니다.
 - 이벤트별 상세 데이터는 `payload(JSONB)`에 저장합니다.
공통 적재 구조를 단순하게 유지하는 데에는 유리하지만, 이벤트별 세부 필드를 기준으로 복잡한 통계나 조건 검색을 수행할 때는 쿼리가 길어지거나 인덱스 최적화에 제약이 생길 수 있습니다.
-